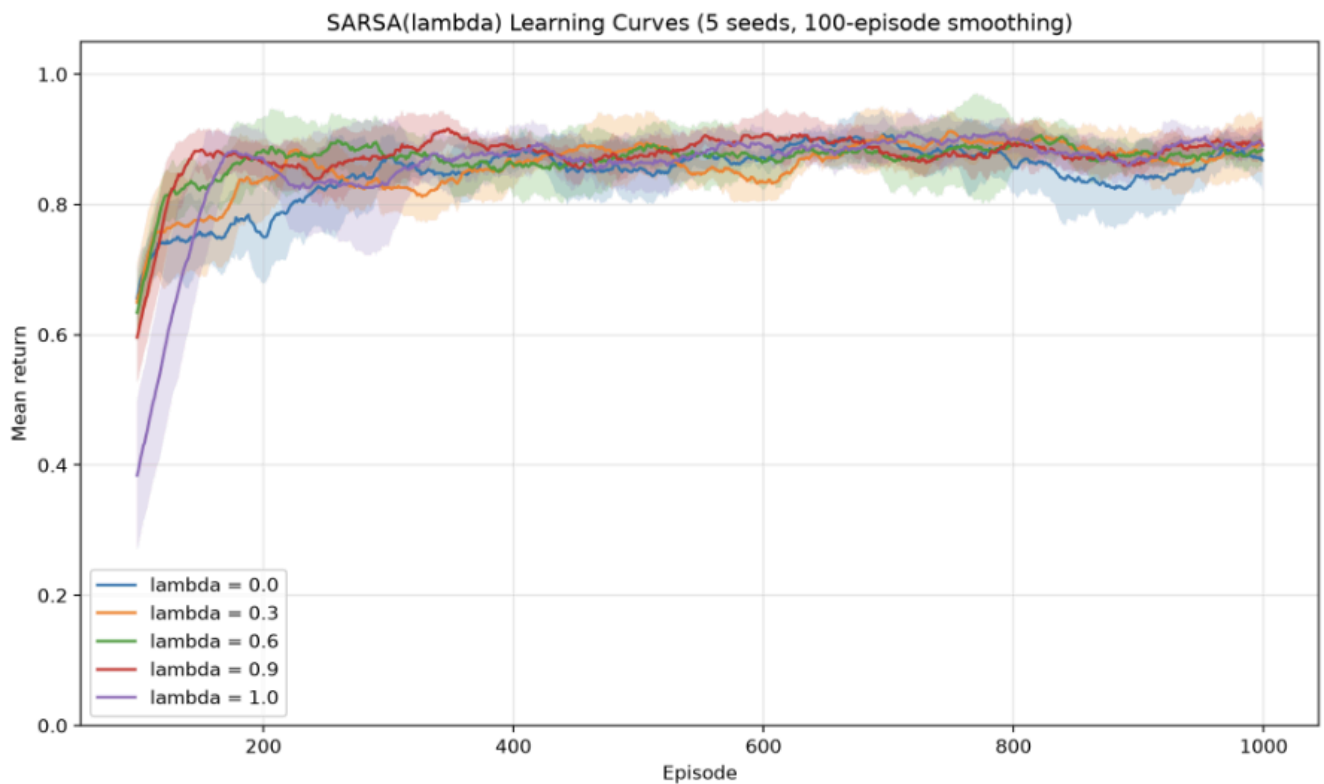


```

S F F F
F H F H
F F F H
H F F G

```

The environment is a 4×4 FrozenLake grid. The agent begins at S and must reach the goal G. The holes marked H end the episode if entered, while frozen tiles marked F are safe. The agent has four actions: left, down, right, and up. Reaching the goal gives a reward of 1, while other transitions provide a reward of 0. Because `is_slippery=False`, the agent moves deterministically in the chosen direction when possible. The observation is a discrete integer representing the agent's position on the grid.



λ	Target return	First episode reaching target	Mean final return
0.0	0.8	222	0.868
0.3	0.8	173	0.892
0.6	0.8	119	0.884
0.9	0.8	123	0.892
1.0	0.8	152	0.890

Each agent was trained for 1,000 episodes. The target return was 0.8. The target-reaching episode was measured using the 100-episode smoothed mean return across five seeds. The mean final return was calculated over the final 100 episodes across all five seeds.

We evaluated the trained greedy policy using $\lambda = 0.9$ and seed 0, with exploration disabled. The agent reached the goal in 6 steps and received a return of 1.0. The episode terminated successfully rather than being truncated.

Why does λ behave this way?

The parameter λ affected learning speed more than final performance. Using a target return of 0.8, $\lambda = 0.6$ reached the target fastest at episode 119, followed closely by $\lambda = 0.9$ at episode 123. This suggests that eligibility traces helped assign credit to earlier state-action pairs that contributed to eventually reaching the goal. With $\lambda = 0$, only the most recent state-action pair was updated, so learning was slower. However, the final returns were similar across all λ values, ranging from 0.868 to 0.892. Very large λ values spread the TD error across many earlier decisions, which can introduce noisier updates. Therefore, λ improved the speed of learning in this environment, but it did not substantially change the final performance.

The Snake environment from Task 1 uses a Box observation space, so it is not directly compatible with the tabular SARSA agent, which requires discrete states. Therefore, we used the discrete FrozenLake environment for Task 2. The SARSA agent was kept environment-independent and was not modified to use Snake-specific state information.